System Test Report

Deliverable 3, Campus Ride-Pooling and Split-Fare System

Prepared by Khushi Ranawat Jira CS455 Deliverable 3 November 2026

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

Result. 48 of 48 end-to-end browser system tests passed in **13** test specification files, executing in **1.4 minutes** with a 100% pass rate against the complete integrated application stack: Next.js 16 frontend and backend, Node.js HTTP/WebSocket server, Socket.IO real-time channels, and a dedicated PostgreSQL 16 database (`crp_test`). The suite validates all core user journeys, multi-role interactions, and edge cases defined in Deliverable 1 requirements.

1. Executive Summary and Scope

Deliverable 3 requires comprehensive system-level validation demonstrating that the complete Campus Ride-Pooling and Split-Fare application operates reliably from an end-user perspective. While the unit test suite (more than 1,500 tests) verifies functions with mocked dependencies, and the route integration test suite exercises isolated API handlers, only browser-based system testing exercises the full vertical stack:

1. **Complete Integration:** Real Chromium browser sessions interacting with real Next.js client and server components, custom Node.js application server (`server.mjs`), Socket.IO WebSocket bi-directional transport, and PostgreSQL 16 database.
2. **Zero Backend Mocking:** No route mocks or database mocks. Every user interaction traverses the real network, invokes real authorization middleware, executes transactional SQL, evaluates database constraints and triggers, and updates the DOM.
3. **Multi-Context Role Interactions:** Cross-role interactions (e.g., rider submitting requests, owner managing pickups, two riders contending for a seat, concurrent Socket.IO messages, administrator moderating complaints) run in separate, isolated browser contexts simultaneously.

2. System Test Architecture and Harness

The system testing framework is built on Playwright Test with TypeScript, structured for deterministic execution, test isolation, and strict reproducibility.

Layer / Component	Implementation Details
Test Runner	Playwright Test (`@playwright/test`) with Chromium engine, headless execution, single worker mode (`workers: 1`) to eliminate inter-test database race conditions.
Application Server	Custom Node.js server (`server.mjs`) initialized via Playwright's `webServer` directive on port 3000, hosting Next.js request handlers, WebSocket server, and notification publishers.
Database Isolation	Dedicated test database `crp_test` running on PostgreSQL 16 (port 5433). Validated with safety guardrails: the fixture refuses non-local hosts, non-`_test` names, and a missing `ALLOW_TEST_DATABASE_RESET` opt-in.
Data Fixtures	Transactional data is truncated before each test while preserving static reference seeds (campus places, transport hubs, vehicle types). Distinct user accounts (Rider, Owner, Admin) are seeded per test with cryptographically valid SHA-256 session cookies injected directly into isolated browser contexts.
Page Object Helpers	Reusable helper abstractions (`RidePageHelper`, `RequestPageHelper`, `NotificationHelper`) ensuring accessible locator queries (`getByRole`, `getByLabel`) rather than brittle CSS selectors.

3. Requirement Traceability

In accordance with the Deliverable 3 traceability framework (PR #93), requirement traceability is centralized in `Deliverables/D3/Traceability_Matrix/`. All browser system tests cite exact Deliverable 1 requirement IDs in their test titles and assertions, enabling automated extraction and linkage into the project-wide traceability

matrix and coverage artifacts.

4. Key Test Suites and Coverage Analysis

4.1 1. Core Rider and Owner Journeys

- **Ride Creation and Search (`rider/ride-creation-and-discovery.spec.ts`):** Validates authenticated owner creating rides with direction, destination hub, pickup point, vehicle type, and departure window. Validates client-side and server-side validation error displays when required fields are missing. Validates rider searching with multi-attribute filters.
- **Join Request Lifecycle (`rider/join-request-lifecycle.spec.ts`):** Validates complete end-to-end state transitions (`pending` → `accepted`, `pending` → `rejected`, `pending` → `withdrawn`, `accepted` → `left`). Proves idempotency keys prevent duplicate database requests when duplicate submissions occur.
- **Trip Progress Management (`owner/ride-management-trip-flow.spec.ts`):** Exercises locking a ride early, cancelling a ride, and the trip sequence Start Pickup → Mark Arrived → Start Trip → Complete Ride. Marking a no-show after the grace period is covered in `cross-role/time-boundaries.spec.ts`.

4.2 2. Safety, Incidents and Governance

- **SOS Emergency Workflow (`cross-role/sos-and-safety.spec.ts`):** Tests raising SOS during active trip states (`pickup_in_progress`, `in_transit`). Confirms `sos_incidents` row generation with unique tracking reference (`SOS-XXXXXXXX`). Validates requirement `FR-RD-15.4`: other pool members are never alerted or exposed to the SOS trigger. Verifies operations admin incident desk workflow: queue ingestion, acknowledgment, resolution with closure note.
- **Admin Workflows and Account Suspension (`admin/admin-workflows.spec.ts`):** Tests configuration management (adding vehicle types and inspecting hubs). Exercises complaint processing from submission to resolution. Validates suspension enforcement: disciplined riders are strictly forbidden from offering rides (HTTP 403 / suspended banner) and submitting join requests. Validates administrative reactivation.

4.3 3. Cross-Role Interactions, Concurrency and Real-Time Services

- **Table T-2 Split-Fare Verification (`cross-role/fare-and-settlement.spec.ts`):** Implements Deliverable 1 Table T-2 whole-rupee splitting: total fare of Rs 500 split across 3 riders produces shares of Rs 167, Rs 167, and Rs 166, with remainder assigned to earliest occupants. Verifies that marked no-shows owe Rs 0.
- **Seat Contention Race (`cross-role/concurrency-seats.spec.ts`):** Two riders simultaneously contend for the single available seat on a capacity-limited vehicle. Verifies optimistic concurrency control: the first accepted rider takes the seat; subsequent acceptance is blocked in the UI and rejected with HTTP 409 at the database level.
- **Socket.IO Real-Time Messaging (`realtime/pool-chat-and-notifications.spec.ts`):** Uses two concurrent browser contexts to verify instant delivery of pool chat messages over WebSockets. Proves namespace/room isolation: riders in ride A cannot eavesdrop on or receive messages from ride B.
- **AI Assistant and Guardrails (`ai/recommendations-and-chatbot.spec.ts`):** Exercises "Ask AI to Suggest" normal ranking. Applies deterministic route-level fault injection for Gemini timeout (< 8s elapsed recovery bound), 503 service outage alerts with retry recovery, and malformed response resilience. Validates strict transactional boundaries: conversational queries asking the bot to book seats or cancel rides are refused with direct manual guidance links.

5. Defect History and Architectural Observations

During system test development, several subtle cross-layer interaction behaviors were analyzed and validated against Deliverable 1 specifications:

1. **Origin Validation (`sameOrigin`):** Mutating endpoints (POST, PATCH) strictly enforce same-origin verification. Browser automation tests dispatching concurrent API requests within a context must execute via `page.evaluate(() => fetch(...))` rather than detached HTTP clients to carry authentic browser headers.
2. **Next.js Route Transition Transitions:** Server component actions invoke `router.refresh()` within `useTransition`. System tests interact after waiting for transition spinners and status label changes, ensuring tests reflect confirmed persistent server state.
3. **Accessible Semantic Selectors:** All tests prioritize accessibility-tree selectors (`getByRole`, `getByLabel`) matching real screen-reader and user interactions rather than implementation-specific HTML classes.

6. Environment and Reproduction

The complete system test suite can be executed locally or via continuous integration:

Database Setup: `docker compose up -d db` starts PostgreSQL; create test database via `createdb -U crp crp_test` and load `apps/web/db/schema.sql`. Set `TEST_DATABASE_URL=postgres://crp:pass@127.0.0.1:5433/crp_test` and `ALLOW_TEST_DATABASE_RESET=crp_test` in `apps/web/.env.local`. Stop any dev server on port 3000 to avoid session collisions.

`npm run test:e2e` (from repository root or `apps/web`)

`npm run test:e2e - apps/web/e2e/tests/admin/admin-workflows.spec.ts` (single file)

CI Pipeline: GitHub Actions workflow `.github/workflows/ci.yml` runs the *System tests (Playwright E2E)* job on every pull request and master branch push.